

Análisis de capacidad

Carga de documentos:

Estas pruebas de carga fueron diseñadas para evaluar los dos flujos más importantes de la aplicación. El primer flujo abarca la etapa de *chunking* y *embedding* del documento, hasta el almacenamiento de los vectores en la base de datos vectorial. El *chunking* consiste en dividir un documento en fragmentos más pequeños (chunks) para facilitar su procesamiento y búsqueda, mientras que el *embedding* transforma cada fragmento en una representación numérica (vector) que captura su significado semántico. El segundo flujo, por su parte, corresponde a la interacción del usuario con el chat en función de los documentos previamente cargados.

Para poder cumplir con este fin se idearon dos escenarios:

Escenario 1 (Carga del documento):

Este escenario no posee un contexto previo, busca probar la crear credenciales y demás recursos para poder lograr la carga y almacenamiento del documento.

- Se crea un usuario, mediante el uso del endpoint de usuarios (El cual se utiliza para crear un usuario)
- Se hace login mediante el endpoint de usuarios/login, donde este me retorna el id del usuario y el token de acceso correspondiente (Importante para poder consumir los endpoints de los microservicios posteriores)
- Se crea un chat para el usuario correspondiente, mediante el uso del endpoint /chats
- Se carga un archivo a este chat mediante el uso del endpoint de chat.

Este Flujo fue creado en Jmeter, y el archivo correspondiente se llama CSV_Data_Set_prompt.jmx dentro de la carpeta Jmeter en el repositorio relacionado al proyecto (<https://github.com/leytoncasta/app-web-notebook-lm>), el cual se ve de la siguiente manera:

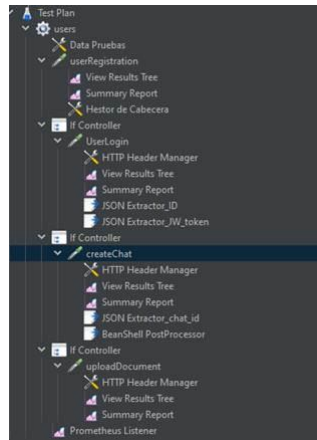


Figura 1. Pruebas Jmeter escenario 1.

Escenario 2 (Preguntas sobre el documento):

Este escenario utiliza los datos que se crearon anteriormente para la carga de documento, guardando el chat id y el token bearer que utilizaremos para la autenticación de la aplicación. Dado esto esta prueba seguirá los siguientes pasos.

- Se cargan los datos antes guardados por el escenario 1.
- Se hace una misma pregunta “¿Qué dice el texto?” para todas las

threads. Todo esto al final se ve de la siguiente manera:

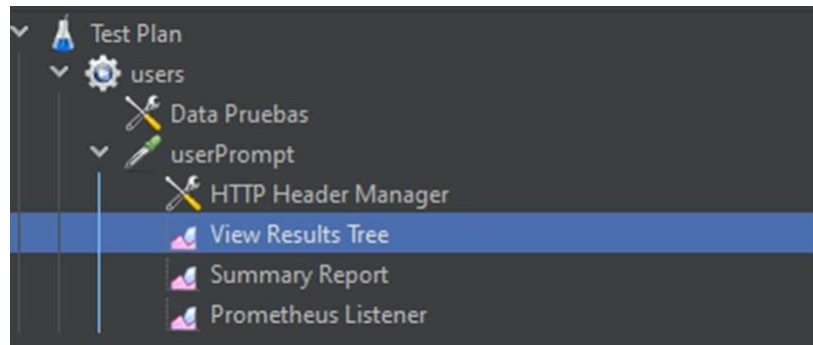


Figura 2. Pruebas Jmeter escenario 2.

Nota:

A estas pruebas se añadió un simple writer para poder generar el histograma que se solicitó en la entrega pasada.

Definición de estado esperado:

Se define el estado óptimo de la aplicación bajo las siguientes condiciones:

Flujo 1

Como usuario concurrente de la aplicación,
quiero poder completar el flujo 1 en menos de 4 segundos incluso cuando hay hasta 2 usuarios accediendo al mismo tiempo,
para asegurar una experiencia fluida y eficiente en condiciones normales de uso.

Flujo 2

Como usuario individual de la aplicación,
quiero recibir respuestas en menos de 1 segundo al utilizar el flujo 2,
para garantizar una interacción rápida y continua en cualquier momento.

En dado caso que cualquiera de estas historias no se cumpla se considerara esto como un estado insuficiente.

Resultados:

Para las pruebas de carga se usó Jmeter, Prometheus y Grafana para poder observar los resultados que implican calcular tiempos de respuesta y la cantidad de solicitudes que terminaron con un status 200 o no.

Dadas estas métricas pasadas a Prometheus se calcularon las siguientes métricas utilizando estas plataformas de observabilidad:

Tasa promedio de throughput por etiqueta (ventana: 1m)

Este indicador que aparece en las gráficas como “Solicitudes exitosas al segundo en el ultimo minuto” se define como la cantidad de solicitudes con código de estado 200 procesadas por minuto por label. Para su cálculo, se configuró en Grafana una tasa (rate) de respuestas exitosas basada en la métrica denominada throughput_total. Como se muestra en la Figura 3, se utilizó el listener de Prometheus, y en la Figura 4 se presenta el código correspondiente en Grafana. Posteriormente, esta métrica fue renombrada y ahora aparece como Solicitudes exitosas por segundo en el último minuto, aunque representa el mismo valor.

```
avg by (label) (rate (throughput_total [1m]))
```

Código 1. Configuración de Grafana.

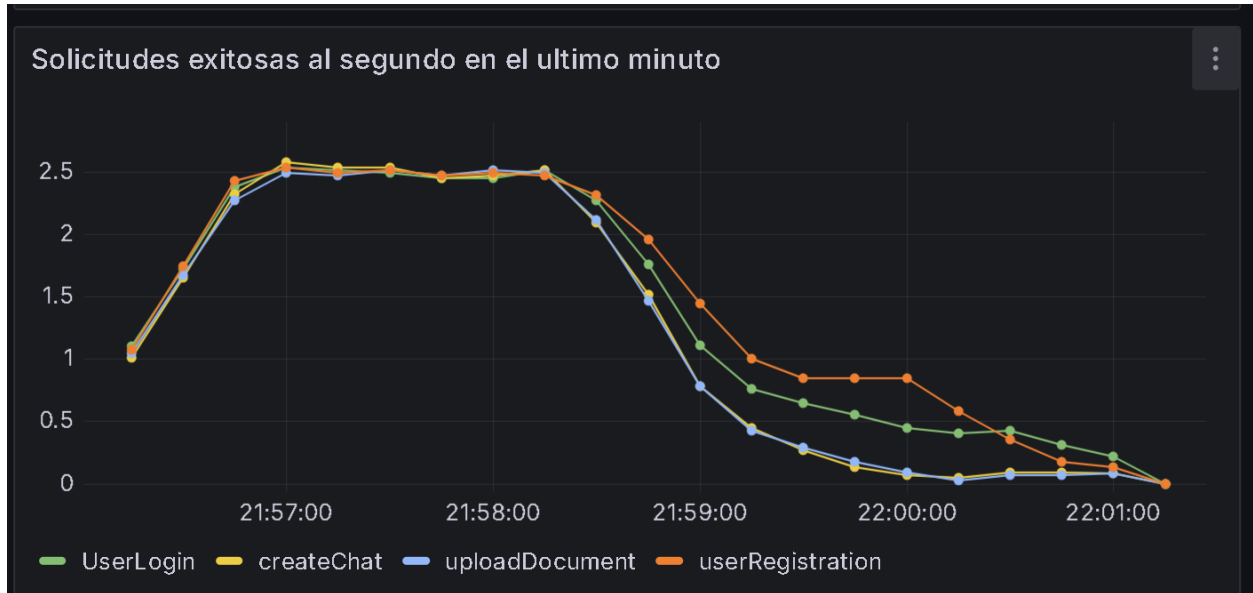


Figura 3. Tasa promedio de throughput por etiqueta (ventana: 1m)

Volumen total de eventos por segundo

Esta métrica es a grandes rasgos lo mismo que “Solicitudes exitosas al segundo generales” la diferencia está en que esta métrica es un agrupado de solicitudes agrupadas por segundo, el código se puede ver en el código 3 y la imagen en la Figura 4.

```
(sum(rate(throughput_total[1m])))
```

Código 3. volumen total de eventos procesados por segundo en el último minuto.

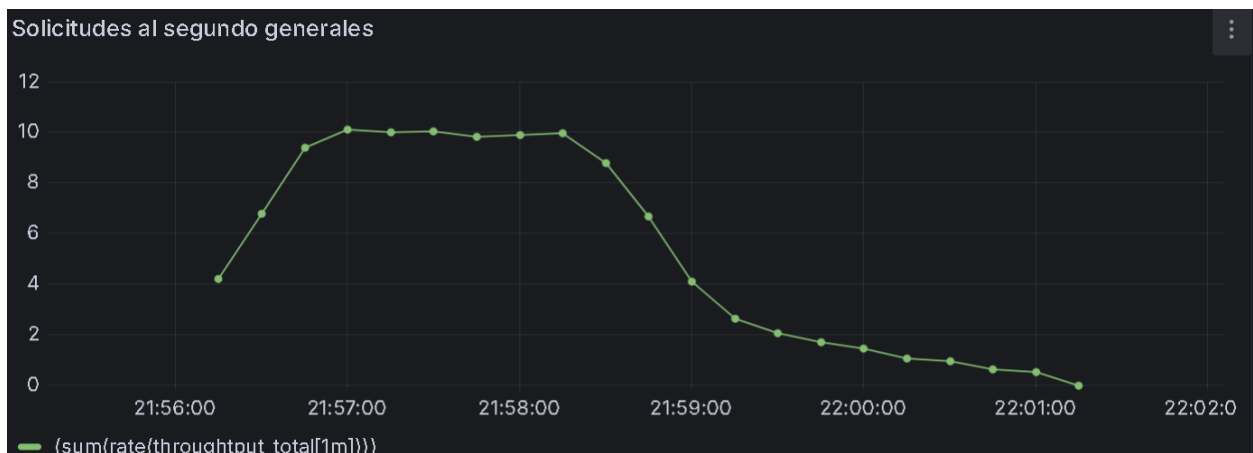


Figura 4. volumen total de eventos procesados por segundo en el último minuto.

Thread number:

En la sección pasada se hizo un intento de estimación de los usuarios concurrentes de la aplicación, pero ahora ya no es necesario porque se cuenta con el número de threads en cada una de las stages (active, finished, started), para saber cuántos usuarios concurrentes tenemos al mismo tiempo haciendo las diversas solicitudes. Esto es más preciso que la métrica pasada, dado que esto nos da el número de threads precisa que finalizaron, activas e iniciando. El código de esta se puede ver en código 4 y se puede ver el tipo de grafica en la figura 5.

```
avg by (state) (avg_over_time(jmeter_threads[1m]))
```

Código 4. Configuración de Grafana de Thread number.

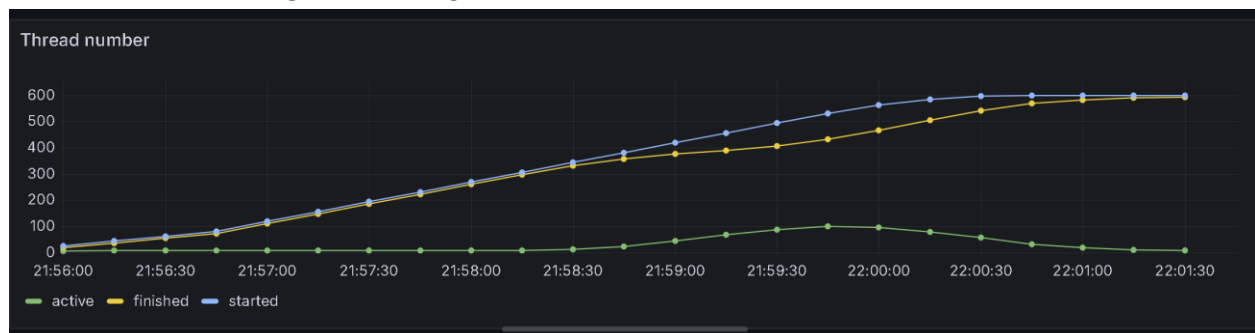


Figura 5. Configuración Grafana Thread number.

Response time:

Esta métrica abarca los tiempos de promedio por minuto de cada tipo de request. Básicamente se trata de los tiempos que se han demorado las solicitudes, dividido la cantidad de solicitudes, como se puede ver en el código 5, y su grafica se puede ver en la Figura 6.

```
avg by (state) (avg_over_time(jmeter_threads[1m]))
```

Código 5. Configuración de Grafana de Response time.

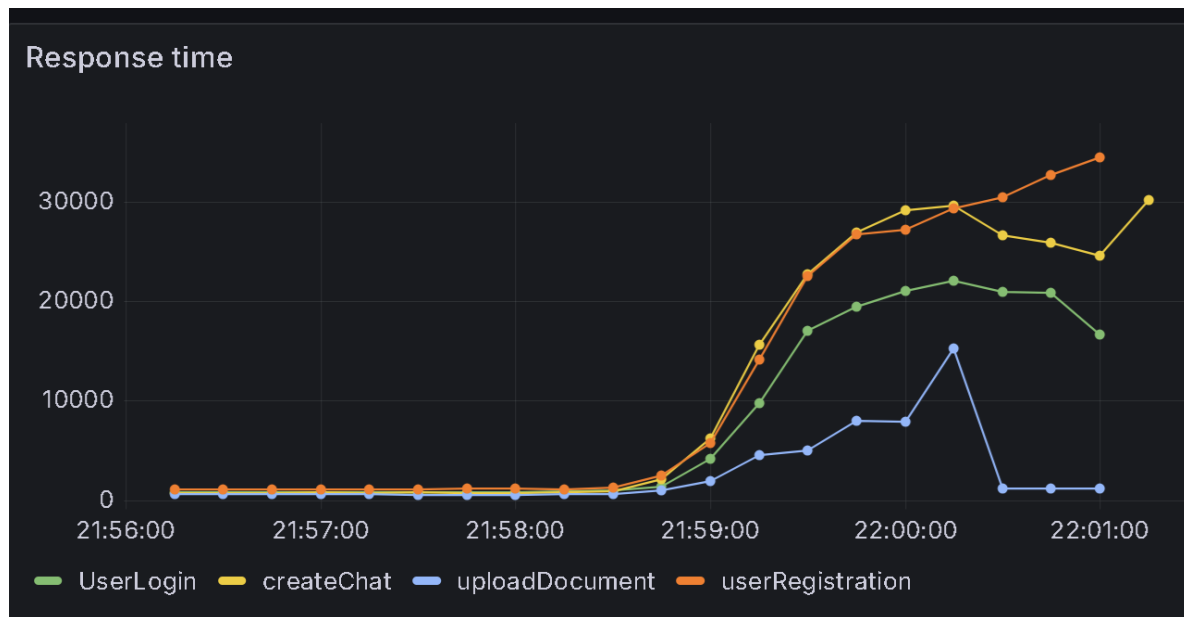


Figura 6. Tiempos de respuesta por endpoint.

Successful per Endpoint:

La métrica "Successful per endpoint" muestra cuántas veces se llamó con éxito a cada endpoint. Agrupa las solicitudes por ruta y suma cuántas se completaron bien, lo que ayuda a ver cuáles partes de tu API son más usadas. El código se puede ver en el **Código 6** y en la figura se puede ver el resultado.

```
sum(throughput_total) by (label)
```

Código 6. Configuración de Grafana de Response time.

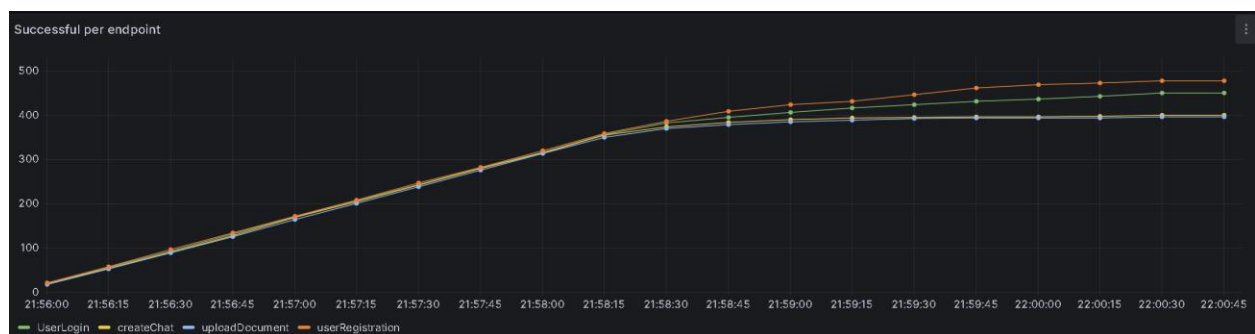


Figura 7. Grafica Successful per Endpoint.

Successful Total:

Es lo mismo que Successful per Endpoint solo que agrupado, el código se puede ver en el código 7 un ejemplo de grafica en la figura 8.

```
sum(throughput_total)
```

Código 7. Configuración de Grafana de Response time.

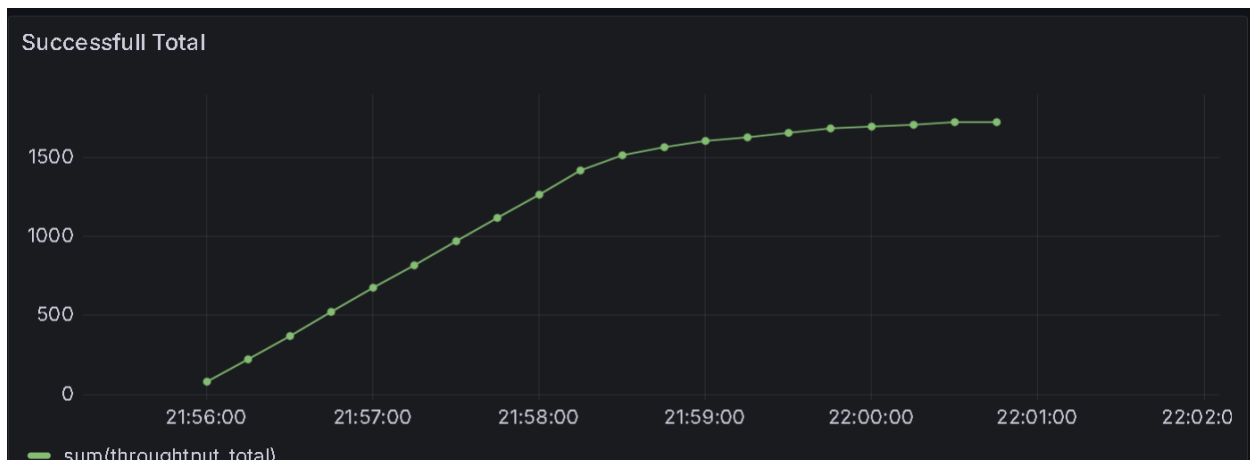


Figura 8. Grafica Successfull Total.

Response time grouped:

Esta métrica abarca los tiempos de promedio por minuto de todas las request sumadas. Básicamente se trata de los tiempos que se han demorado las solicitudes, dividido la cantidad de solicitudes, como se puede ver en el código 8, y la grafica se puede ver en la Figura 9.

```
sum(  
  rate(response_time_summary_sum[1m])  
  /  
  rate(response_time_summary_count[1m])  
)
```

Código 8. Configuración de Grafana de Response time.

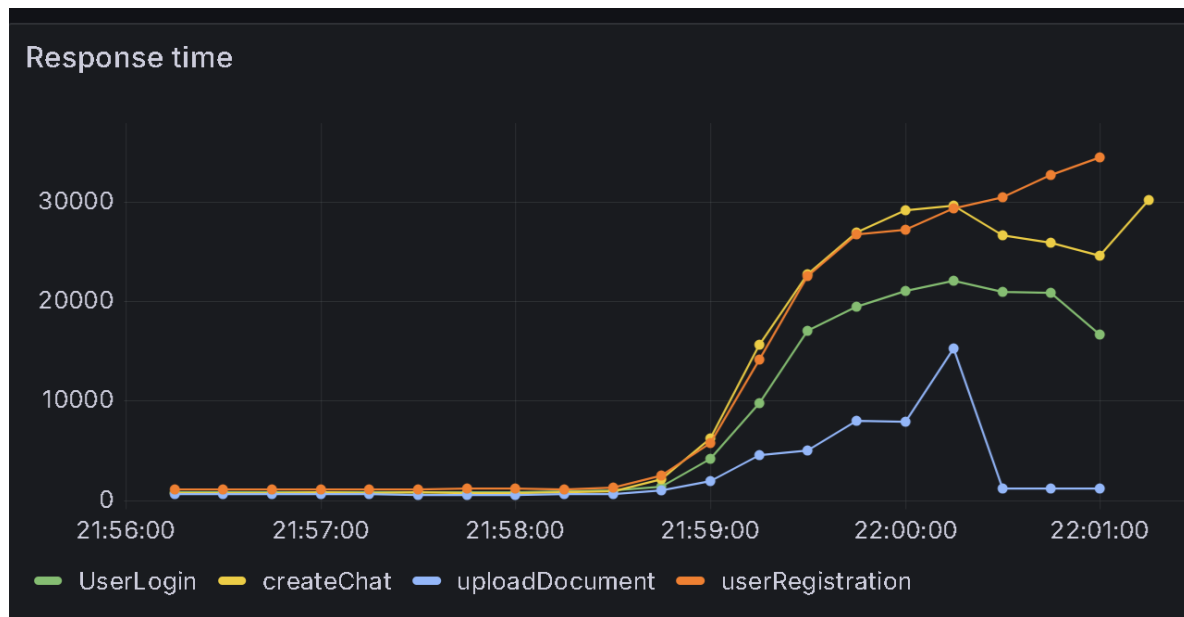
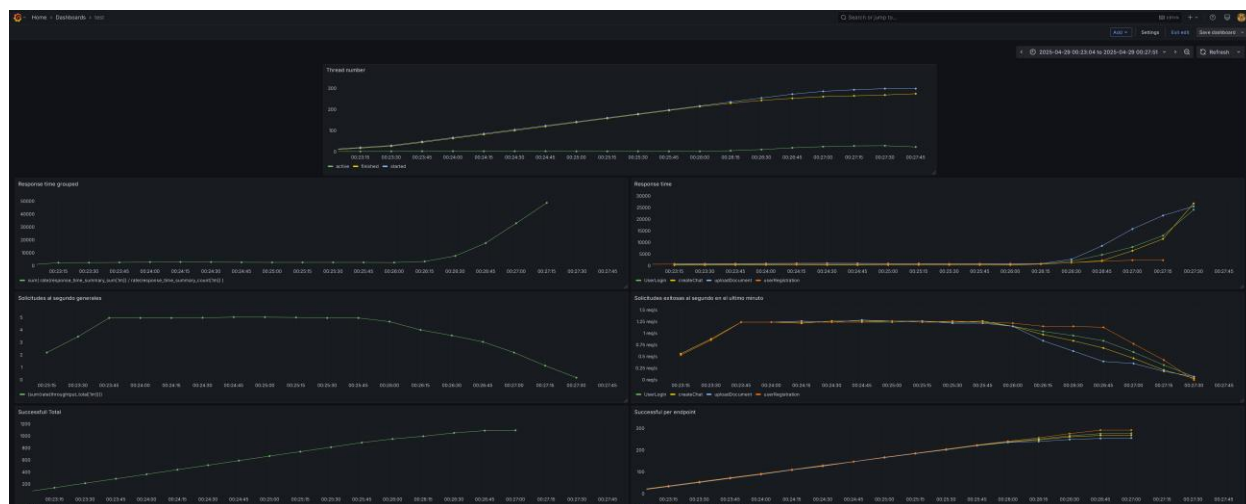


Figura 9. Tiempos de respuesta por endpoint.

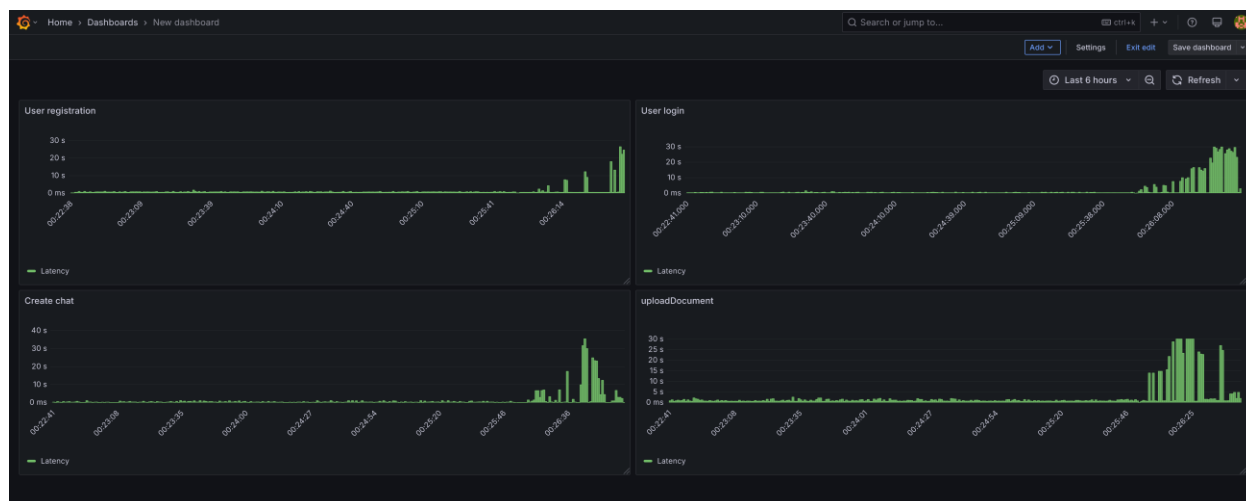
Parte A:

Se tiene que aclarar que tal como se mencionó en el documento perteneciente a la entrega de la planeación del análisis de carga (entrega pasada), se enviaron **300 hilos (threads)** distribuidos en un período de **240 segundos**, utilizando un único ciclo (*loop*). Por lo tanto, todos los resultados que se presentan a continuación corresponden a esa configuración de prueba. Se utilizo esta configuración de forma preliminar para seguir con el hilo conductor de las anteriores pruebas de carga, dado que las anteriores pruebas de carga comprendían un total de threads se 150 en 120 segundos, lo único que se está haciendo es manteniendo la razon threads segundos y aumentando los resultados para poder visualizar de mejor forma.

Resultados Escenario 1:



Al analizar el número de hilos concurrentes, se observa que, antes de la caída, el sistema era capaz de manejar normalmente las solicitudes entrantes. Esto sugiere que la aplicación puede sostener de manera estable hasta 3 hilos simultáneos. Asimismo, la arquitectura logra responder aproximadamente 5 solicitudes por segundo en condiciones no sobrecargadas. La falla de los servicios parece haber comenzado a las 00:26:00, momento en el que el throughput comienza a disminuir mientras los tiempos de respuesta aumentan, indicando un deterioro generalizado del sistema. En promedio, cada solicitud presenta un tiempo de respuesta cercano a 1 segundo. Si agrupamos todos los endpoints y estimamos el tiempo que tarda en procesarse un flujo completo, este se aproxima a los 3 segundos por hilo en condiciones normales.



Los gráficos presentados son histogramas que muestran la latencia de cada solicitud a lo largo del tiempo. A través de ellos, se puede observar la **consistencia temporal de las**

métricas de latencia, lo que permite identificar qué endpoints presentan mayor cantidad de solicitudes con alta latencia y cuán densa es dicha latencia.

Se observa que los **picos de latencia del endpoint "create chat"** superan a los de cualquier otro, lo que sugiere que, si se desea optimizar el sistema, este endpoint debería ser la primera prioridad para análisis y mejoras.

Por otro lado, aunque el endpoint **"user registration"** presenta algunos picos, estos parecen ser mayormente *outliers*, por lo que, entre todos los endpoints analizados, es el que requiere menor atención en términos de rendimiento.

Como conclusión, y considerando tanto la magnitud como la densidad de las latencias, el orden de prioridad para posibles optimizaciones debería ser el siguiente:

1. create chat
2. upload document
3. user login
4. user registration

CPU:

En la arquitectura A, durante la ejecución del flujo 1, los *workers* alcanzaron un pico de utilización del **36%**, lo que indica una **subutilización** de los recursos disponibles en ese componente.

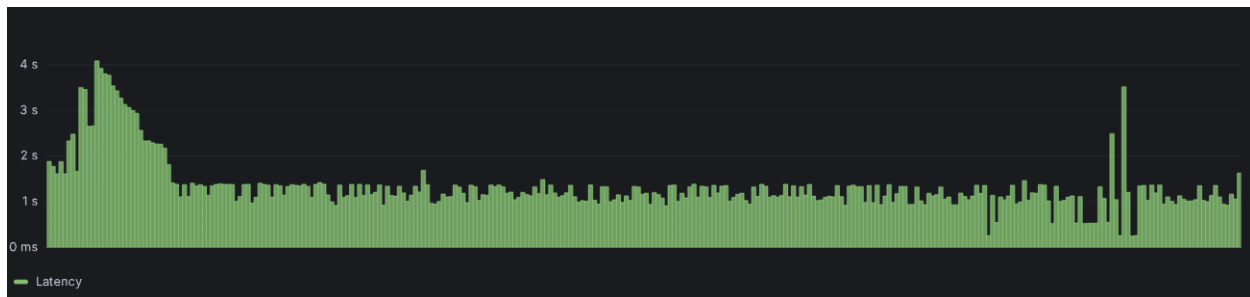
En contraste, el componente **web** presentó una utilización máxima del **322%**, lo cual excede ampliamente su capacidad esperada. Este valor indica que el servidor web **colapsó bajo la carga**, evidenciando un cuello de botella crítico en esa parte de la arquitectura.

Por ende las pruebas de carga fueron exitosas.

Resultados Escenario 2:



Con respecto a los resultados se tienen 1.5 Treads por unidad de tiempo activas, y un tiempo de respuesta promedio de 1,3 s, respondiendo 1,3 solicitudes al segundo y teniendo todas las solicitudes exitosas por ende podemos estimar que este no es el límite superior de la arquitectura y por ende esta arquitectura está cumpliendo para le path 2 y estas especificaciones.



Con respecto al histograma de tiempo y latencia, se observa que los valores de los tiempos de respuesta coinciden con los mostrados en la gráfica de promedios. En general, el comportamiento es mayormente estable, y el único pico significativo al inicio del gráfico corresponde al momento en que se estaban creando las máquinas, por lo que no representa un problema de rendimiento sostenido.

CPU:

En la arquitectura A, durante la ejecución del flujo 2, los *workers* alcanzan un pico de utilización de tan solo **38.96%**, lo que indica que están **subutilizados** y existe un margen considerable para aumentar la carga sin comprometer el rendimiento.

Por otro lado, en el escenario 2 del componente web, la utilización máxima observada es del **50%**, lo cual también sugiere que este componente **no está operando a su máxima capacidad**.

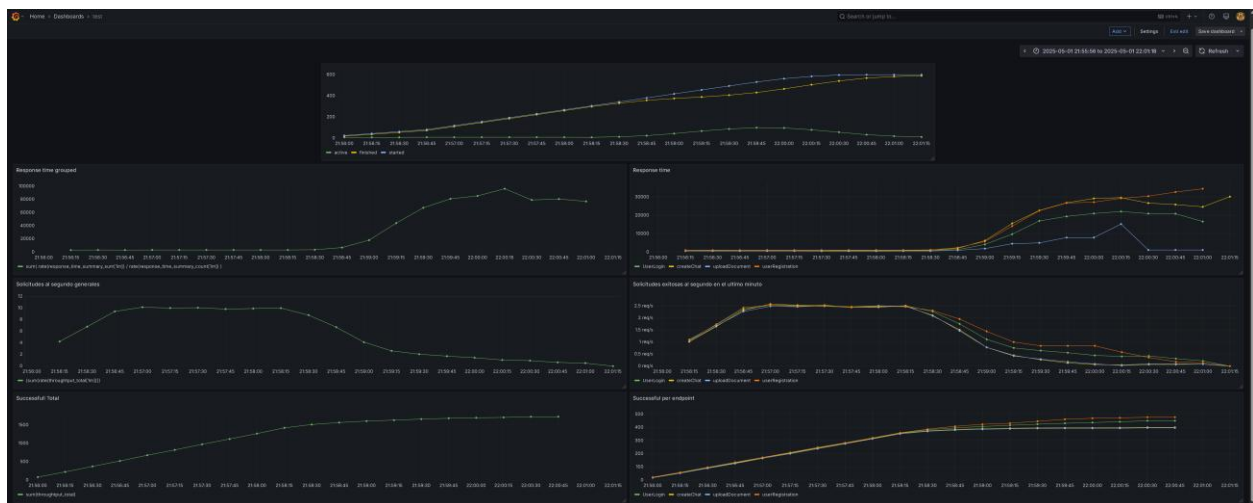
Por ende este flujo no tiene ningún problema para correr en la máquina.

Parte B:

Para la parte b se lanzaron 600 threads para el escenario 1y 1748 para el escenario 2 en 240 segundos ambas.

Resultados Escenario 1:

En el escenario 2, se estima —a partir del punto en que comienza a disminuir el throughput— que la aplicación puede soportar entre 8 y 12 usuarios concurrentes en condiciones normales. En cuanto a los tiempos de respuesta agregados, se observa que el sistema mantiene un comportamiento estable con un tiempo promedio de 3 segundos por solicitud antes de que se produzca una degradación en el rendimiento. El sistema logró atender un total de 10 solicitudes por segundo con código de estado 200. Además, la capacidad de manejo de solicitudes concurrentes con respuesta exitosa (status 200) se duplicó con respecto al número de usuarios, mientras que el número de hilos concurrentes se triplicó, reflejando una mejora significativa en la capacidad de concurrencia.



Con respecto al histograma de usuarios en determinados horarios, sí se observa una tendencia general al aumento en los tiempos de respuesta. En particular, las operaciones de *user registration*, *create char* y *user login* presentan los mayores incrementos, lo que podría indicar cuellos de botella en esos procesos. En contraste, la operación *upload document* no muestra problemas significativos, lo cual se correlaciona con un uso más

estable de la CPU por parte de los workers.



CPU:

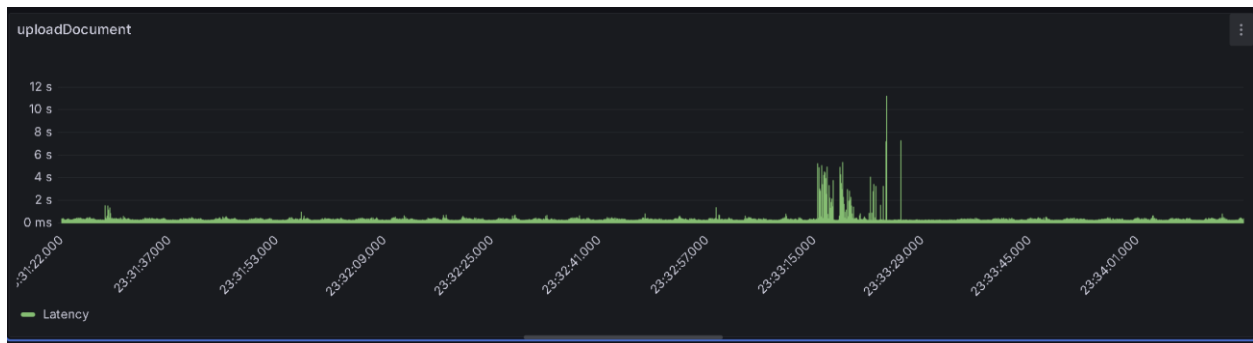
En este escenario, los *workers* alcanzaron un pico máximo de uso de CPU del 197%. Por su parte, los servidores web registraron un consumo del 397%, lo que refuerza la hipótesis de que estas máquinas superaron su capacidad máxima estimada del 300%. Esto sugiere que, como es habitual, los *workers* están altamente demandados, mientras que los servidores web representan el principal cuello de botella del sistema.

Resultados Escenario 2:

En cuanto al número de solicitudes respondidas por segundo, el sistema alcanza un rendimiento de 10 solicitudes por segundo y 4 hilos activos en ese escenario, con un tiempo de respuesta promedio de 0.3 segundos. Este tiempo de respuesta es tres veces mejor que el observado en la arquitectura A, flujo 2.



En cuanto a los histogramas mostrados en la parte inferior, se observa que algunos tiempos de respuesta alcanzan hasta 12 segundos; sin embargo, estos casos son outliers. El comportamiento general del conjunto no indica una tendencia sostenida hacia tiempos de respuesta cada vez mayores. Por lo tanto, no se puede atribuir este aumento puntual a una degradación del sistema, en especial considerando que, posteriormente, los tiempos de respuesta vuelven a valores normales, lo que evidencia una recuperación del servicio.



CPU:

En este escenario, los *workers* alcanzaron un pico de uso de CPU del 150%, mientras que los servidores web registraron un máximo del 52%. Esto indica que ambos servicios están siendo subutilizados. Con base en la utilización observada de los *workers*, se estima que el número máximo de solicitudes que podrían procesarse a plena capacidad es de aproximadamente 3,496 en un período de 240 segundos.

Conclusiones:

Al comparar las distintas arquitecturas, se observa que la incorporación de un *instance group* junto con colas produce una mejora significativa en el rendimiento del sistema. En particular, los tiempos de respuesta se reducen hasta en un **factor de cuatro**, y la mayoría de las métricas muestran mejoras de al menos **tres veces**.

Un ejemplo destacable es el **path 2 en la arquitectura B**, que fue capaz de manejar de forma estable **1,748 solicitudes**. Considerando que los *workers* presentan un pico de 150 solicitudes cada uno, se estima que esta arquitectura puede **soportar hasta 3,496 solicitudes en 240 segundos**, lo que representa un aumento de capacidad desde **10 hasta 20 solicitudes por segundo**. Esta capacidad de respuesta se considera bastante elevada y refleja una mejora sustancial en la escalabilidad del sistema.

Como se puede observar en relación con los estados definidos al inicio del documento, ambas arquitecturas cumplen con los tiempos de respuesta mínimos requeridos bajo condiciones normales de ejecución de la aplicación. En general, se estima que ambas

arquitecturas soportan adecuadamente hasta 2 usuarios concurrentes. Por lo tanto, ambas soluciones pueden considerarse aprobadas para el flujo 1, siendo la única diferencia relevante la cantidad de usuarios que alcanzan el punto de carga de documentos (*upload document*).

En cuanto al flujo 2, la arquitectura A no cumple con los tiempos de respuesta establecidos, ya que estos superan el límite de 1 segundo. En contraste, la arquitectura B sí cumple con dicho requerimiento, por lo que se recomienda su adopción para este caso.